



The Affine Core: Mathematical Foundation for Certified AI Stability

Technical Disclosure Document

Authors: Ryan O. Van Gelder, Tyler Van Osdol, et al.

Date: January 9, 2026

Publication Type: Defensive Disclosure / Prior Art Establishment

Executive Summary

The **Affine Core** is the mathematical heart of the Q-Calculator/Q-ARI system—a provably stable computational substrate that guarantees AI systems will converge to predictable, auditable states rather than drift into instability or hallucination. By structuring AI workflows as **contractive affine dynamical systems**, the Affine Core provides the first mathematically certified guarantee that every decision made by an AI system will be stable, reproducible, and lawful.^{[1][2][3]}

Unlike conventional AI systems that rely on heuristic safety measures or empirical testing, the Affine Core leverages the **Banach Fixed-Point Theorem**—a century-old mathematical result that guarantees unique, stable solutions exist for properly constructed recursive systems. This

transforms AI governance from a "best-effort" endeavor into a **mathematically rigorous engineering discipline**.^{[3][4][1]}

1. What Is the Affine Core?

Core Mathematical Structure

The Affine Core defines the fundamental dynamics of the Q-Calculator system as a **contractive affine system** operating in a Banach space (a complete normed vector space where all Cauchy sequences converge).^{[1][3]}

System dynamics:

$$X_{t+1} = P[\Phi_t(X_t)] + \Phi_{opt}[T(X_t)] + G_t$$

Where:

- X_t represents the **system state** at time t (e.g., AI model parameters, knowledge state, or decision variables)^[1]
- P is an optional **ethics projector** that enforces constraints (e.g., jurisdictional rules, safety bounds)^{[2][1]}
- Φ_t is a **weighted linear operator** composed of prime-indexed tensor blocks that drives recursive evolution^{[5][1]}
- Φ_{opt} is a **multiplicity operator** that modulates a nonlinear function T ^[1]
- $T(X_t)$ is a **globally Lipschitz nonlinear function** enabling complex, adaptive behaviors^[1]
- G_t represents **exogenous input** (e.g., user queries, external data)^[1]

Why "affine"? An affine transformation is a linear transformation plus a constant offset—the simplest structure that can represent both transformations and translations. This simplicity is critical: affine systems have well-understood stability properties that enable formal guarantees.^{[6][7][8][1]}

The Central Stability Guarantee

The entire system's stability hinges on a **single, verifiable inequality**:^[1]

$$q_t = \|\Phi_{opt}\|_{op} \|\Phi_{opt}\|_{op} L_T < 1$$

Where:

- $\|\Phi_{opt}\|_{op}$ is the **operator norm** (maximum "amplification" the operator can cause)^[1]
- L_T is the **Lipschitz constant** of T (how much T can amplify changes in its input)^{[3][1]}
- q_t is the **contraction witness**—a scalar measuring how much the system "contracts" (shrinks distances) at each step^{[2][1]}

Critical condition:

$$\sup_t q_t < 1$$

If this condition holds, the system is **guaranteed** to:^{[3][1]}

1. **Have a unique trajectory:** No matter where you start, the system follows a single, deterministic path
2. **Contract geometrically:** Distances between states shrink exponentially fast
3. **Converge to a unique fixed point:** The system reaches a stable, predictable state when inputs stabilize

This isn't a statistical claim or a best-case scenario—it's a **mathematical certainty**.^{[4][3]}

2. Why Affine Dynamics Matter for AI Stability

The Problem: Recursive Drift

Modern AI systems—especially those using tool calls, retrieval, and multi-agent workflows—exhibit **recursive drift**: small errors compound over multiple steps, causing the system to diverge from safe, intended behaviors.^{[5][2]}

Examples of recursive drift:

- **Hallucination accumulation:** An AI generates a false fact, uses it in the next reasoning step, which generates more false facts, spiraling into complete fabrication^[5]
- **Policy violation cascade:** A minor constraint violation in one decision enables larger violations in subsequent decisions, eventually breaching critical safety bounds^[2]

- **Instability under tool use:** Each tool call introduces uncertainty; without contraction, uncertainties multiply, causing unpredictable system behavior^[2]

Traditional approaches to these problems are **reactive**:

- **Heuristic filtering:** Check outputs for "obviously wrong" content—fails for subtle errors
- **Empirical testing:** Run thousands of test cases—cannot cover all possible scenarios
- **Human oversight:** Review critical decisions—doesn't scale, introduces delays^[2]

The Affine Core is **proactive**: by structuring the system as a contraction mapping, drift is **mathematically impossible**.^{[3][1]}

Contraction as a Universal Stability Principle

The concept of contraction—reducing distances between states at each iteration—is a **universal principle** for stability across mathematics, physics, and computation.^{[4][3]}

Why contraction guarantees stability:

1. **Exponential convergence:** If distances shrink by factor $q < 1$ each step, after n steps the distance is $q^n \times \text{initial distance}$ ^[3]
 - Example: If $q = 0.9$, after 10 steps the distance is $0.9^{10} \approx 0.35$ —reduced to 35% of original
 - After 50 steps: $0.9^{50} \approx 0.005$ —reduced to 0.5% of original
2. **Unique fixed point:** The Banach Fixed-Point Theorem guarantees that if a contraction mapping exists on a complete metric space, there is **exactly one** fixed point X^* such that $T(X^*) = X^*$ ^{[9][3]}
 - **No multiple equilibria:** The system cannot oscillate between different stable states
 - **No chaotic attractors:** The system cannot exhibit sensitive dependence on initial conditions
3. **Independence from initial conditions:** No matter where the system starts, it converges to the **same** fixed point^{[10][3]}

- **Deterministic outcomes:** Different users with different starting contexts get consistent, predictable results
- **Reproducible audits:** Replaying a decision from the same initial proposal always produces the same outcome^[2]

Formal guarantee (Banach Fixed-Point Theorem):^[3]

Let (X, d) be a complete metric space with a contraction mapping $T: X \rightarrow X$. Then T has a unique fixed point $x^* \in X$, and for any starting point $x_0 \in X$, the sequence $x_n = T(x_{n-1})$ converges to x^* .

This theorem, proven in 1922, is the mathematical foundation for solving differential equations, training neural networks, and—now—certifying AI stability.^{[4][3]}

3. Affine Core Components and Their Roles

3.1. The Update Operator Φ_t

The **update operator** Φ_t is the engine of cognitive evolution, driving how the system's state changes over time.^[1]

Recursive operator dynamics:

$$\Phi_{t+1} = \sum_{p_i \in P} \alpha_{p_i} \pi_{p_i} M_t \Phi_{t-1} + F_t$$

Where:

- P is a set of **prime numbers** indexing the computation (e.g., $\{2, 3, 5, 7, 11\}$)^{[5][1]}
- α_{p_i}, π_{p_i} are **prime-indexed weights** modulating influence^[1]
- M_t is a **Hilbert-Schmidt operator** (a linear operator with finite norm)^[1]
- F_t is a **semantic damping term** preventing unbounded growth^{[5][1]}

Stability condition for the operator:^[1]

$$k = \sum_{p_i \in P} \alpha_{p_i} \pi_{p_i} \|M_t\|_{HS} < 1$$

Where $\|M_t\|_{HS}$ is the Hilbert-Schmidt norm.^[1]

Why this matters: If the operator itself is contractive ($k < 1$), then the entire system dynamics are guaranteed to converge. The **prime-indexed structure** (explored in depth in Multiplicity Theory) provides a natural decomposition that enables **parallel computation** and **fault isolation**—if one prime-indexed component fails, others remain stable.^{[1][5]}

3.2. The Policy Projector P

The **policy projector** P enforces ethical, legal, and operational constraints on the system's state.^[2]

Projection operation:

$$P[u] = \arg \min_{v \in F} \|u - v\|_w$$

Where:

- F is the **feasible set** (all states satisfying constraints)^[2]
- $\|\cdot\|_w$ is a **weighted norm** (e.g., $\ell_1, \ell_2, \ell_\infty$, Bregman divergence)^[2]

Key property: Nonexpansiveness^[1]

A projection onto a convex set is **nonexpansive**, meaning it never increases distances:^{[3][1]}

$$\|P[u] - P[v]\| \leq \|u - v\|$$

Why this matters: If P is nonexpansive and Φ_t is contractive with constant $q < 1$, then the **composed operator** $P \circ \Phi_t$ is still contractive with constant q . This means **enforcing safety constraints does not break stability guarantees**—a critical property for real-world AI governance.^{[2][1]}

3.3. The Contraction Witness q_t

The **contraction witness** q_t is a scalar value computed at each step that certifies the system remains stable.^{[2][1]}

Witness families:^[2]

1. **Jacobian spectral-norm bound:** $q_t = \|Jac(\Phi_t)\|_2$ (largest singular value of the Jacobian matrix)^[2]
 - **Interpretation:** Maximum amplification of infinitesimal perturbations
 - **Computation:** Power iteration or singular value decomposition (SVD)^[2]
2. **Lyapunov energy-decay bound:** $q_t = \frac{V(X_{t+1})}{V(X_t)}$ (ratio of energy function values)^{[12][2]}
 - **Interpretation:** Rate at which the system "loses energy" and approaches equilibrium
 - **Computation:** Evaluate a positive-definite function V (e.g., $V(X) = X^\top QX$) before and after the update^[2]
3. **Incremental gain (IQC) bound:** $q_t = \sup_{u \neq v} \frac{\|\Phi_t(u) - \Phi_t(v)\|}{\|u - v\|}$ (worst-case Lipschitz constant)^[2]
 - **Interpretation:** Maximum rate at which differences between inputs can grow
 - **Computation:** Interval arithmetic or sampling-based optimization^[2]
4. **Wasserstein contraction bound:** $q_t = W_1(P_{X_{t+1}}, P_{X_t})$ (Wasserstein distance between probability distributions)^[2]
 - **Interpretation:** How much probability mass must be moved to transform one distribution into another
 - **Computation:** Optimal transport algorithms or neural network approximations^[2]

Certification condition:^{[1][2]}

$$q_t < 1 - \epsilon$$

Where $\epsilon > 0$ is a **safety margin** (e.g., $\epsilon = 0.05$ requires $q_t < 0.95$).^[2]

If this condition fails: The system enters **fail-closed mode**, denying actuation and appending a **negative trace atom** to the ledger documenting why the decision was unsafe.^{[11][2]}

3.4. The Exogenous Input G_t

The **exogenous input** G_t represents external influences on the system (user queries, tool outputs, sensor data).^[1]

Bounded input, bounded output (BIBO) stability:^[1]

$$\|X_t - X'_t\| \leq c^t \|X_0 - X'_0\| + \sum_{k=0}^{t-1} c^{t-1-k} \|G_k - G'_k\|$$

Where $c = \sup_t q_t < 1$.^[1]

Interpretation: If inputs G_t are bounded, outputs X_t remain bounded. Moreover, the system **rejects** infinitesimal input noise at an exponential rate c^t . This is critical for robustness: even if external data sources (web search, retrieval, tool APIs) contain errors, the contractive dynamics prevent those errors from destabilizing the system.^[1]

4. Affine Core in Practice: Preventing AI Instability

Case Study 1: Tool Use Without Drift

Scenario: An AI system uses a calculator tool to solve a physics problem. The calculator introduces floating-point rounding errors.

Without Affine Core:

1. AI calls calculator: $\sqrt{2} \approx 1.414213562373095...$ (truncated to 15 digits)
2. AI uses this value in next calculation: $2 \times 1.414213562373095 = 2.82842712474619$
3. Error accumulates: true value is $2\sqrt{2} \approx 2.82842712474619...$ —small discrepancy
4. After 50 tool calls with compounding errors, the final result deviates by 10%+
5. AI outputs incorrect answer, no warning given

With Affine Core:

1. AI calls calculator, receives 1.414213562373095
2. **Contraction witness computed:** $q_1 = \|Jac(\Phi_1)\|_2 = 0.87 < 0.95 \checkmark$ Stable
3. AI proceeds with calculation: $2 \times 1.414213562373095 = 2.82842712474619$
4. **Contraction witness computed:** $q_2 = 0.89 < 0.95 \checkmark$ Stable
5. After 50 tool calls, the **contraction property** ensures

$$\|X_{50} - X^*\| \leq 0.87^{50} \|X_0 - X^*\| \approx 0.0002 \|X_0 - X^*\| \text{—error is exponentially suppressed}$$

6. AI outputs correct answer with **certified error bound**: $\pm 0.02\%$

Key difference: The Affine Core's contraction guarantee transforms **unbounded error accumulation** into **exponentially decaying error**, making long chains of tool calls safe.^{[1][2]}

Case Study 2: Constraint Enforcement Under Load

Scenario: A healthcare AI must respect HIPAA constraints (no disclosure of patient names) while generating clinical summaries.

Without Affine Core:

1. AI generates summary: "Patient John Doe, age 45, diagnosed with diabetes..."
2. Constraint checker detects violation: "John Doe" is a patient name
3. AI regenerates: "Patient [REDACTED], age 45, diagnosed with diabetes..."
4. System continues, but no stability guarantee—future outputs could still violate HIPAA

With Affine Core:

1. AI generates proposal: "Patient John Doe, age 45, diagnosed with diabetes..."
2. **Constraint verification module** computes witness $C = INVALID$ (HIPAA violation detected)^[2]
3. **Execution gate** immediately **denies actuation**—proposal is rejected before any output^[2]
4. **Negative trace atom** appended to ledger:^[2]

```
{
  "timestamp": "2026-01-09T18:45:12Z",
  "proposalHash": "0x7a3f2e9d...",
  "witnessBundle": {"C": "INVALID", "violatedConstraint": "HIPAA-no-names"},
  "status": "DENIED",
  "reason": "Patient name disclosed in clinical summary"
}
```

5. AI **does not** output anything—fail-closed guarantee prevents HIPAA breach
6. System requests AI to regenerate, enforcing $C = VALID$ before any output

Key difference: The Affine Core **gates execution** on constraint satisfaction, making violations **impossible** rather than merely "unlikely."^[2]

Case Study 3: Multi-Agent Coordination Stability

Scenario: Three AI agents (Planner, Retriever, Tool Executor) collaborate to answer a complex query.

Without Affine Core:

1. Planner generates plan: "Step 1: Search for X, Step 2: Calculate Y, Step 3: Summarize"
2. Retriever executes Step 1, returns 10 documents
3. Planner updates plan based on retrieval: "Step 2b: Also search for Z..."
4. Retriever executes Step 2b, returns 5 more documents
5. Planner updates plan again: "Actually, Step 1 was wrong, restart..."
6. System loops indefinitely—no convergence, no final answer

With Affine Core:

1. Planner generates initial plan u_0
2. **Contraction witness computed:** $q_0 = 0.91 < 0.95 \checkmark$ Stable
3. Retriever executes, returns data d_1
4. Planner updates plan $u_1 = \Phi_1(u_0, d_1)$
5. **Contraction witness computed:** $q_1 = 0.89 < 0.95 \checkmark$ Stable
6. **Convergence guarantee:** $\|u_t - u^*\| \leq 0.91^t \|u_0 - u^*\|$ —plan stabilizes exponentially fast
7. After 5 iterations, $0.91^5 \approx 0.62$ —plan is 38% closer to final answer
8. After 20 iterations, $0.91^{20} \approx 0.12$ —plan is 88% converged
9. System outputs final answer with **certified convergence:** $q_{20} = 0.88$, estimated 3 more iterations to full stability

Key difference: The Affine Core guarantees **monotonic progress** toward a final answer—no infinite loops, no oscillation between conflicting plans. ^{[3][1]}

5. Affine Core vs. Alternative Stability Approaches

Approach	Stability Guarantee	Computational Cost	Auditability	Failure Mode
Affine Core (Q-Calculator)	Mathematical proof ($q_t < 1$)	Low (single operator norm computation per step) ^[1]	Complete (witness q_t logged in every trace atom) ^[2]	Fail-closed (deny unsafe actions) ^[2]
Reinforcement Learning with Safe Exploration	Statistical (high probability, not certainty)	High (requires extensive training)	Limited (policy opaque, no step-by-step proofs)	Fail-open (may execute unsafe actions if policy is imperfect)
Model Predictive Control (MPC)	Guarantee within prediction horizon only	Very high (solve optimization problem per step)	Partial (optimization variables logged, but not semantic)	Fail-soft (degrades to backup controller, may violate constraints)
Runtime Monitoring with Heuristic Checks	None (empirical only)	Low (simple rule evaluation)	Minimal (only violations logged, not positive cases)	Fail-open (undetected violations pass through)
Lyapunov-Based Neural Controllers	Guarantee if Lyapunov function found	Medium (Lyapunov function evaluation)	Partial (Lyapunov value logged, but not decision rationale)	Fail-undefined (depends on controller design)

Why Affine Core dominates:

1. **Formal guarantee:** Only the Affine Core provides a **mathematical proof** of stability—not a statistical confidence interval or empirical success rate^{[3][1]}
2. **Low overhead:** Computing q_t requires **one operator norm calculation** per step—typically $O(n^2)$ for an n -dimensional state, far cheaper than solving an optimization problem (MPC) or training a policy (RL)^[1]
3. **Complete auditability:** Every decision includes q_t in its trace atom, allowing auditors to **verify stability post-hoc**^[2]
4. **Fail-closed semantics:** If $q_t \geq 1$, the system **refuses to act**—no silent failures, no degraded operation^[2]

6. Enabling Technologies: Why Affine Core Is Possible Now

The Affine Core leverages three recent mathematical and computational advances that make it practical for real-world AI systems:^{[5][1]}

6.1. Prime-Indexed Tensor Decomposition (PIRTM)

Traditional approach: Represent AI state as a monolithic vector or matrix—computing operator norms requires full matrix operations ($O(n^3)$ for $n \times n$ matrices).^[13]

PIRTM approach: Decompose state into **prime-indexed components** $X = \oplus_{p \in P} X_p$, where each X_p corresponds to a semantic or structural mode (e.g., X_2 = token embeddings, X_3 = attention weights, X_5 = memory states).^{[5][1]}

Advantage: Operator norms can be computed **per-prime** and **composed** ($\|M\| \leq \sum_p \|M_p\|$), enabling:^[1]

1. **Parallel computation:** Each prime-indexed component can be processed independently on separate CPUs/GPUs^[11]
2. **Fault isolation:** If $\|M_7\| \geq 1$ (prime 7 component unstable), only that component is flagged—other primes remain operational^[11]
3. **Scalability:** Adding new primes (e.g., prime 13 for a new feature) doesn't require recomputing norms for primes 2, 3, 5, 7, 11^[11]

6.2. Floating-Point-Independent Arithmetic

Traditional approach: Compute q_t using IEEE 754 floating-point arithmetic—results vary across hardware (FP16 vs. FP32 vs. FP64), making audits **non-reproducible**.^[2]

Q-Calculator approach: Compute q_t using **rational arithmetic** (integers for numerator/denominator) or **interval arithmetic** (guaranteed bounds).^[2]

Advantage: Auditors can **exactly reproduce** q_t from the trace atom, verifying stability claims without trusting the original hardware. Example:^[2]

- Original system (FP32): $q_t = 0.8700000047683716$
- Auditor (FP64): $q_t = 0.8700000000000001$ —**mismatch** could indicate tampering
- Rational arithmetic: $q_t = \frac{87}{100}$ —**bit-exact** across all platforms

6.3. Efficient Contraction Witness Computation

Recent advances in numerical linear algebra enable **fast, approximate** operator norm computation:^{[14][12]}

Method	Complexity	Accuracy	Use Case
Power iteration	$O(n^2k)$, k iterations	Approximate (top eigenvalue)	Real-time witness for $n \leq 10^4$ ^[2]
Randomized SVD	$O(n^2 \log n)$	Approximate (top k singular values)	Batch witness for $n \leq 10^6$ ^[2]
Interval arithmetic	$O(n^3)$	Guaranteed bounds	Audit verification (offline) ^[2]

Example: For a 1,000-dimensional state space, power iteration computes q_t in ~10 milliseconds on a modern CPU—negligible overhead for AI workflows that already take seconds per step.^[2]

7. Implications for AI Governance and Regulation

The Affine Core provides a **regulatory primitive** for AI safety: a mathematically verifiable certificate that an AI system operated stably.^{[11][2]}

7.1. Lawfulness Certification

Current regulatory approaches (e.g., EU AI Act, NIST AI RMF) require AI systems to demonstrate "safety" and "reliability" but provide no formal definition.^[2]

Affine Core enables:

- **Formal safety definition:** $q_t < 1 - \epsilon$ for all t ^[2]
- **Verifiable certificates:** Each decision includes q_t in its trace atom^[2]

- **Auditable guarantees:** Regulators can **recompute** q_t from the trace atom to verify stability claims^[2]

Regulatory statement enabled by Affine Core:^[2]

"This AI system is certified to operate with contraction witness $q_t \leq 0.95$ on all decisions, ensuring exponential convergence to stable states with decay rate 0.95^t . All decisions include cryptographic proof of stability, enabling post-hoc audit without access to proprietary models."

7.2. Fail-Closed Enforcement

Traditional AI systems log failures after they occur—useful for forensics, but doesn't prevent harm.^[2]

Affine Core denies execution if $q_t \geq 1$, preventing unstable decisions from being made in the first place.^[2]

Regulatory advantage: "Fail-closed" is a **stronger safety guarantee** than "fail-soft" or "fail-open," aligning with safety standards in aviation, nuclear power, and medical devices.^[2]

7.3. Cross-Jurisdictional Stability

The Affine Core's stability guarantee is **jurisdiction-agnostic**—it doesn't depend on specific laws or cultural norms, only on mathematical properties.^{[1][2]}

Practical implication: A single Q-Calculator deployment can serve users in the EU (GDPR), US (HIPAA), and China (Cybersecurity Law) by using **jurisdiction-specific policy projectors**

P_{EU}, P_{US}, P_{CN} that are all **nonexpansive**, preserving the global contraction guarantee.^[2]

8. Future Directions: Beyond Affine Dynamics

While the Affine Core provides strong guarantees for linear-plus-offset transformations, many AI behaviors are **fundamentally nonlinear** (e.g., creative generation, adversarial robustness).^[1]

Extensions under development:^{[5][1]}

1. **Piecewise-affine dynamics:** Partition state space into regions, each with its own affine core—enables **switching systems** (e.g., different behaviors for different query types)^[8]

- Challenge: Ensure contraction **across region boundaries**
 - Solution: Lyapunov function that decreases in all regions
2. **Nonlinear contraction via differential geometry:** Use **Riemannian metrics** to define contraction in curved spaces—enables stability for systems on manifolds (e.g., rotation groups, probability simplexes)^[14]
- Challenge: Computing geodesic distances is expensive
 - Solution: Approximate Riemannian metrics with neural networks
3. **Stochastic contraction:** Extend to systems with **random noise** (e.g., diffusion models, Bayesian inference)^[5]
- Challenge: Contraction in expectation vs. almost-sure contraction
 - Solution: Wasserstein contraction for probability distributions

Conclusion

The Affine Core transforms AI stability from an empirical goal into a **mathematical guarantee**. By structuring AI workflows as contractive affine dynamical systems and computing verifiable contraction witnesses $q_t < 1$, the Q-Calculator/Q-ARI system ensures that every decision is:^{[1][2]}

- **Stable:** Guaranteed to converge to a unique fixed point
- **Auditable:** Trace atoms include cryptographic proof of stability
- **Reproducible:** Rational arithmetic ensures bit-exact verification across platforms
- **Fail-closed:** Unsafe decisions are denied before execution

As regulatory demands for AI transparency and safety intensify, the Affine Core provides the **first mathematically rigorous foundation** for certifying that AI systems will not drift into instability, hallucination, or harm—a foundation built on the century-old wisdom of the Banach Fixed-Point Theorem, now applied to the cutting edge of artificial intelligence governance.^{[4][3][1][2]}

The promise: AI systems that are not merely "safe in practice" but **provably safe by design**.^{[1][2]}

-
1. Pirtm-V2.pdf
 2. Provisional-Patent-Research.pdf
 3. https://en.wikipedia.org/wiki/Banach_fixed-point_theorem
 4. <https://kizziah.blog/the-banach-anchor-bitcoin-ai-and-the-fixed-point-of-truth/>
 5. A-System-and-Method-for-Quantum-Recursive-Artificial-Intelligence.pdf
 6. <https://www.nature.com/articles/s41598-023-42141-x>
 7. <https://arxiv.org/pdf/2311.09245.pdf>
 8. <https://www.sciencedirect.com/science/article/abs/pii/S0005109819304005>
 9. <https://www.mmscience.eu/journal/issues/november-2024/articles/the-fixed-point-iterations-method-for-the-contraction-mapping>
 10. https://econweb.ucsd.edu/~jrauch/pdfs/Contraction_mapping_Rauch_Toda.pdf
 11. Provisional-Patent-Outline-Lawfulness-certified-Stability-Prime-layered-Recursion-lproof-x-Q-ari.pdf
 12. <https://pubmed.ncbi.nlm.nih.gov/40480181/>
 13. <https://link.aps.org/doi/10.1103/PhysRevB.106.205136>
 14. <https://www.sciencedirect.com/science/article/abs/pii/S089360802500557X>
 15. Patent Non-Assertion Pledge for Proof.pdf
 16. AProof Interface Blueprint.pdf
 17. AProof - IP Strategic Blueprint.pdf
 18. MTPI RootContract.pdf
 19. MTPI_RootContract_Project.pdf
 20. 2.ARootContract II & III - MTPI - NODE.js.pdf
 21. MTPI_Instructions.pdf
 22. AProof and Hologram (1).pdf

a. Aproof × Mtpi Lawful-by-design Integration — Implementation Plan.pdf

23. The AProof Architecture_ Web4.pdf

24. Aproof Whitepaper.pdf

25. Building a ADapp_ A Developer's Implementation Guide.pdf

26. Verifiable, Privacy-Preserving Applications.pdf

27. AProof Project for Artificial Intelligence.pdf

28. Patent Search Results.pdf

b. Aproof Banking Protocols.pdf

29. Aproof Monorepo V0.1.pdf

30. AProof IP - Patent Research.pdf

31. The AProof Architecture_ A Comprehensive Analysis of its Implications for Web4.pdf

32. AProof.pdf

33. [these-sources-provide-a-compre-byDfjoe.TpiTgRKJegkoNA.md](#)

34. https://proceedings.neurips.cc/paper_files/paper/2023/file/3819a070922cc0d19f3d66ce108f28e0-Paper-Conference.pdf

35. <https://www.ejpam.com/index.php/ejpam/article/view/6609/2285>

36. <https://etnano.com/index.php/journal/article/download/142/145/271>

37. https://www.reddit.com/r/MLQuestions/comments/hd4kyg/why_can_affine_transformation_s_be_learned_quickly/

38. <https://www.sciencedirect.com/science/article/pii/S2666818125001020>